

Politechnika Rzeszowska
Wydział Matematyki i Fizyki Stosowanej

Sprawozdanie

Alorytmy i struktury danych

ZADANIE PROJEKTOWE/ALGORYTMICZNE

Autor:
MATEUSZ DZIAŁOWSKI

20 stycznia 2026

Spis treści

1	Wstęp	2
2	Opis problemu	3
3	Podstawy teoretyczne	3
3.1	Złożoność algorytmu	3
4	Szczegóły implementacji	3
4.1	Analiza problemu	3
4.2	Rozwiązanie problemu - podejście pierwsze	3

1 Wstęp

Celem niniejszego opracowania projektu jest przedstawienie implementacji oraz analizy wydajnościowej algorytmu, rozwiązującego problem wyszukiwania specyficznych podciągów w zadanym ciągu liczb całkowitych dodatnich wejściowych. Dokumentacja obejmuje szczegółowy opis problemu, podstawy teoretyczne, implementację algorytmu oraz wyniki testów wydajnościowych przeprowadzonych na różnych zestawach danych wejściowych.

2 Opis problemu

Dla zadanego ciągu liczby całkowitych dodatnich ciągu $A[]$ o długości N , należy znaleźć wszystkie podciągi o długości $M = 5$ (pięcioelementowe) w danym ciągu, które spełniają kryterium: Suma elementów na pozycji 2 i 4 jest większa od sumy elementów na pozycji 1, 3 i 5.

Dla, podciągu $T[] = [t_0, t_1, t_2, t_3, t_4]$, warunek przyjmuje postać:

$$t_1 + t_3 > t_0 + t_2 + t_4 \quad (1)$$

Przykład: Dla ciągu $A[] = [1, 130, 1, 9, 11, 6, 1, 1, 1, 3, 1]$, zgodnie z powyższym kryterium, podciągi spełniające warunek to:

- $T[M] = [1, 130, 1, 9, 11]$, ponieważ $(130 + 9 > 1 + 1 + 11)$
- $T[M] = [1, 9, 11, 6, 1]$, ponieważ $(9 + 6 > 1 + 11 + 1)$
- $T[M] = [1, 1, 3, 1, 1]$, ponieważ $(1 + 1 > 1 + 3 + 1)$

Jak widać, w powyższym przykładzie istnieją trzy podciągi spełniające zadane kryterium.

3 Podstawy teoretyczne

Algorytm opiera się na technice przesuwania okna o stałej $M = 5$. Dla każdego możliwego podciągu o długości M w ciągu $A[]$, algorytm sprawdza, czy spełnia on warunek określony w równaniu (1). Jeśli tak, podciąg jest zapisywany do wyniku.

3.1 Złożoność algorytmu

Teoretyczna analiza złożoności algorytmu przedstawia się następująco:

- **Złożoność czasowa:** Sprawdzenie jednego okna zajmuje czas stały $O(1)$. Ponieważ okno przesuwamy przez cały ciąg o długości N , wykonujemy $N - M + 1$ operacji, M jest stałą co powoduje że teoretyczna złożoność powinna wynosić $O(N)$.
- **Złożoność obliczeniowa:** Algorytm wymaga przechowywania ciągu wejściowego o wielkości N oraz tablicy wyników M , która jest stałą. Równanie wygląda następująco: $O(N + M)$, co teoretycznie w najgorszym przypadku, gdy każdy podciąg spełnia warunek złożoność pamięciowa wynosi $O(N)$.

4 Szczegóły implementacji

4.1 Analiza problemu

Choć algorytm jest stosunkowo prosty, jego implementacja wymaga uwzględnienia kilku kluczowych aspektów, takich jak efektywne zarządzanie pamięcią, optymalne podejście przy wykorzystaniu pętli oraz optymalizacja operacji sprawdzania warunków, co może znacząco wpłynąć na wydajność końcowego rozwiązania. Pod uwagę trzeba wziąć również obsługę różnych przypadków brzegowych, takich jak bardzo krótkie ciągi wejściowe lub ciągi nie spełniające żadnego z kryteriów co wciąż wpływa na wydajność implementacji.

4.2 Rozwiązanie problemu - podejście pierwsze

W podejściu pierwszym, postąpimy w sposób bezpośredni, iterując przez każdy możliwy podciąg o długości M w ciągu $A[N]$ i sprawdzając, czy spełnia on warunek określony w równaniu (1). Jeśli tak, dodajemy go do listy wyników.

1. Dane wejściowe.

Naszymi danymi wejściowymi są:

- Ciąg liczb całkowitych dodatnich $Tab[]$ również,
- długość ciągu N ,

- stała $M = 5$,

2. Dane wyjściowe.

Naszymi danymi wyjściowymi są:

- Tablica wyników `Wynik[]`, zawierająca wszystkie podciągi

Implementacja wykorzystuje zagnieżdżone pętle iteracyjne, wraz z zmiennymi sterującymi, aby efektywnie przeszukiwać ciąg wejściowy i identyfikować podciągi spełniające określone kryteria. Poniżej przedstawiono szczegółowy opis działania poszczególnych pętli:

- **Pętla zewnętrzna (i)** - przesuwa okno przez cały ciąg (od 0 do $N-4$), że względu na warunek sprawdzający (1) musimy pętlę ograniczyć do $N-4$, w celu uniknięcia błędów. Przedstawionej pętli sprawdzamy krok po kroku każdy możliwy podciąg o długości $M = 5$, czy spełnia on warunek z równania (1), również pętla przesuwa okno o jeden element do przodu w każdej iteracji w celu sprawdzenia następnych możliwych podciągów.
- **Pętla wewnętrzna (j)** - rozpoczyna się od pozycji (i) i iteruje do $N-4$, sprawdzając warunek z równania (1) dla każdego podciągu zaczynającego się na pozycji i .
- **Pętla iteracyjna (k)** - przechodzi przez elementy podciągu (od 0 do $M-1$)
- **Pętla kopiująca (m)** - przepisuje spełniający warunek podciąg do tablicy wyników

Gdy warunek z równania (1) zostanie spełniony dla danego okna, algorytm kopiuje 5 elementów do tablicy wyników i przerywa wewnętrzne pętle poprzez ustawienie wartości sterujących ($j = N, k = M$), co wymusza przejście do następnej pozycji okna.